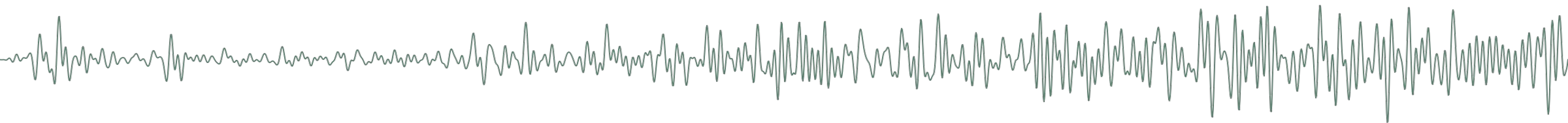




AI-BASED CLASSIFICATION OF EMG SIGNALS FOR

Hand Gesture Classification Analysis

Methods · Features · Parameters · Comparison of Six Models



Burak Talha Güçlü | MSc Thesis Work

Presentation Outline

1 Problem, EMG and motivation

3 Signal preprocessing

5 Six models: parameters & rationale

7 Results and detailed comparison

9 Challenges and recommendations

2 Datasets and hand gestures

4 Feature selection and meanings

6 Evaluation methodology

8 Cylindrical effect & per-class analysis

10 Conclusion

Why EMG Hand-Gesture Recognition?

EMG (electromyography) measures the electrical activity produced when a muscle contracts. Decoding which gesture is performed from forearm surface signals is a critical problem for prosthetics, rehabilitation and human-machine interfaces.



Prosthetic hand control

Natural control for amputees



Rehabilitation

Muscle-activation tracking and therapy



Human-machine interface

Touchless commands / interaction

CORE CHALLENGE

The signal varies from person to person, even from trial to trial.

Similar grips (cylindrical ↔ closed) produce very close muscle activation.

So the choice of features and model is decisive.

BACKGROUND

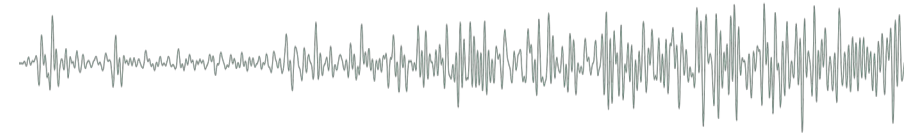
What Is a Surface EMG Signal?

Surface EMG (sEMG) electrodes measure the electrical activity produced when the forearm muscles contract. Different gestures activate different muscle groups → distinguishable signal patterns.

- Non-invasive, real-time, low-cost
- Noisy; varies by person and trial
- The raw waveform is high-dimensional → summarized by features
- Time-domain features are cheap and effective

KEY IDEA

Instead of feeding the raw waveform to the model, we split it into short windows and extract compact descriptors of muscle activity. The classifier maps these descriptors to a gesture.



OVERVIEW

The Study's Pipeline

Goal: compare six models on two datasets fairly, using one and the same pipeline.



Every step is fixed for all models → the only source of differences is the model itself.

Two Complementary Datasets

NinaPro DB5

Open-source benchmark database

Channels	8 (Myo armband)
Subjects	S1, S2, S5, S10
Gestures	7 classes
Sampling	200 → 1000 Hz
Features	32 dims/window
Size	19,591 windows

Custom Data (Ecem & Selman)

Low-cost 2-channel recording

Channels	2
Subjects	Ecem, Selman
Gestures	5 classes
Sampling	1000 Hz
Features	8 dims/window
Size	67,885 windows

NinaPro DB5 — Details

A standard sEMG benchmark. The first 8 channels of the Myo armband are used. The 200 Hz raw signal is upsampled to 1000 Hz for temporal resolution. B- and C-series gestures are taken from the E2 and E3 exercise files.

8

EMG channels

4

subjects

7

gestures

19.591

window

Why NinaPro? Standard, multi-subject and reproducible; results are comparable with the literature.

Exercise A			Exercise B			Exercise C			Exercise D						
1	Index flexion		1	Thumb up		1	Large diameter grasp		16	Quadpod grasp		1	Flexion of the little finger		
2	Index extension		2	Extension of index and middle, flexion of the others		2	Small diameter grasp (power grip)		17	Lateral grasp		2	Flexion of the ring finger		
3	Middle flexion		3	Flexion of ring and little finger, extension of the others		3	Fixed hook grasp		18	Parallel extension grasp		3	Flexion of the middle finger		
4	Middle extension		4	Thumb opposing base of little finger		4	Index finger extension grasp		19	Extension type grasp		4	Flexion of the index finger		
5	Ring flexion		5	Abduction of all fingers		5	Medium wrap		20	Power disk grasp		5	Abduction of the thumb		
6	Ring extension		6	Fingers flexed together in fist		6	Ring grasp		21	Open a bottle with a tripod grasp		6	Flexion of the thumb		
7	Little finger flexion		7	Pointing index		7	Prismatic four fingers grasp		22	Turn a screw (grasp the screwdriver with a stick grasp)		7	Index of index and little finger		
8	Little finger extension		8	Adduction of extended fingers		8	Stick grasp		23	Cut something (grasp the knife with an index finger extension grasp)		8	Flexion of ring and middle finger		
9	Thumb adduction		9	Adduction of extended fingers		9	Writing tripod grasp		Rest 						
10	Thumb abduction		10	Wrist supination (axis: middle finger)		10	Power sphere grasp								
11	Thumb flexion		11	Wrist pronation (axis: middle finger)		11	Three finger sphere grasp								
12	Thumb extension		12	Wrist supination (axis: little finger)		12	Precision sphere grasp								
			13	Wrist pronation (axis: little finger)		13	Tripod grasp								
			14	Wrist flexion		14	Prismatic pinch grasp								
			15	Wrist extension		15	Tip pinch grasp								
			16	Wrist radial deviation											
			17	Wrist ulnar deviation											
			17	Wrist extension with closed hand											

DATASET 2

Custom Data — Ecem & Selman

Collected from two subjects with a low-cost 2-channel setup. Signals are stored per channel (ch1, ch2) for each gesture across four .mat files.

2

EMG channels

2

subjects

5

gestures

67.885

window

Limitation: Only 2 channels → limited spatial information to separate similar grips. This is the main reason behind later performance gaps.

Hand Gestures Recognized (7 Movements)

1. B1: Thumb Up
2. B2: Scissors
3. B5: Open
4. B6: Closed
5. B7: Point
6. C5: Cylindrical
7. C14: Pinch

Selected Hand Gestures

NinaPro (7)

- Thumb up (B1)
- Scissors (B2)
- Open (B5)
- Closed (B6)
- Point (B7)
- Cylindrical (C5)
- Pinch (C14)

Custom Data (5)

- point
- rock
- closed
- cylindrical — most-confused class**
- open

Signal Preprocessing — Parameters

NinaPro

- Repetitions are selected
- Upsampling 200 $\rightarrow$ 1000 Hz ($\times 5$)
- Window: 150 samples (150 ms)
- Step: 30 samples $\rightarrow$ 80% overlap
- Output: 32-dim CSV + raw (150 $\times$ 8) tensor

Custom Data

- Onset: mean |amplitude| > 435 over a 40 ms window
- 150 ms windows from the onset
- Step: 30 ms $\rightarrow$ 80% overlap
- 2 channels $\times$ 4 features = 8
- Output: custom_dataset.csv

KEY CONCEPT

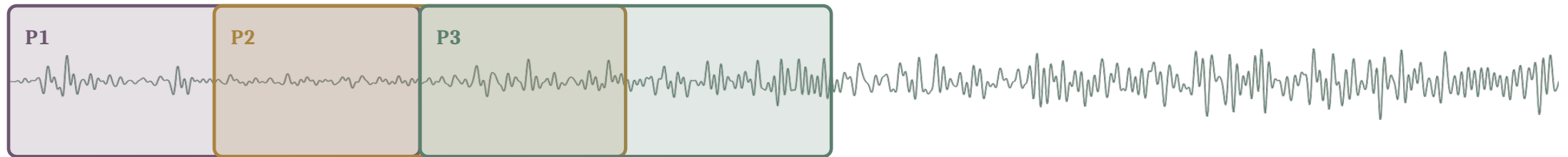
What Exactly Is a 'Window'?

Window is a short, fixed-length segment cut from the continuous EMG signal (150 ms here). The smallest unit the model classifies is not the whole signal but a single window.

- One feature vector (RMS/MAV/WL/SSC) is extracted per window
- The model predicts each window separately → one gesture yields many windows
- Sliding window: the window advances by a fixed step (30 ms)
- 80% overlap: consecutive windows largely coincide → more samples, temporal continuity

IN SHORT

1 window = 1 feature vector = 1 prediction. Classification is done on these small pieces; the final gesture decision comes from combining the windows.



Window width 150 ms · step 30 ms · overlap 80% (sliding, overlapping windows)

Why Do We Upsample 200 Hz → 1000 Hz?

NinaPro is recorded at 200 Hz; in preprocessing it is upsampled 5× (to 1000 Hz). The goal is NOT to add new signal information — there are three practical reasons:

1

Enough samples / window

At 200 Hz, 150 ms = 30 samples; at 1000 Hz, 150 samples. Difference-based WL and SSC become far more stable.

2

Single pipeline

The custom set is already 1000 Hz with 150-sample windows. Upsampling lets the same window + feature code apply to both.

3

CNN input length

The 1D CNN expects a length-150 raw input; 30 samples are too few for large kernels (10, 5).

IMPORTANT NOTE

Upsampling is interpolation — it adds no real new information or frequency (the Nyquist limit of 200 Hz is 100 Hz; anything above is already lost). **Gain: more stable features + consistency across both datasets + a suitable input length for the CNN.**

Feature Selection — Why the Time Domain?

- Very low compute cost → suitable for real-time / embedded prosthetics
- Robust to the small shifts and noise of surface EMG
- Well-established and proven for gesture recognition in the sEMG literature
- Reduces a 150-sample window to 4 numbers per channel with little information loss
- Lets simple, data-efficient classifiers reach high accuracy

SELECTION PRINCIPLE

The four features complement each other: amplitude (RMS, MAV), complexity (WL) and frequency (SSC) — four distinct pieces of information with little redundancy. Frequency-domain features are costlier and were deemed unnecessary here.

Features I — RMS and MAV

RMS

Root Mean Square

$$\text{RMS} = \sqrt{\left(\frac{1}{N} \sum x_i^2 \right)}$$

The root-mean-square value of the signal. Represents the total power/energy the muscle produces; the stronger the contraction, the higher it is.

MAV

Mean Absolute Value

$$\text{MAV} = \left(\frac{1}{N} \sum |x_i| \right)$$

Mean absolute amplitude. The most classic sEMG amplitude estimator; reflects the signal's intensity and is slightly more robust to noise than RMS.

Features II — WL and SSC

WL

Waveform Length

$$WL = \sum |x_{i+1} - x_i|$$

Sum of absolute differences between consecutive samples — the cumulative waveform length. Indirectly measures signal complexity (amplitude + frequency combined).

SSC

Slope Sign Changes

$$SSC = \sum [(x_i - x_{i-1})(x_i - x_{i+1}) < 0]$$

Counts how many times the slope sign changes (number of peaks/valleys). Estimates the signal's frequency content; higher frequency → more sign changes.

Feature Vector and Dimensionality

Each window is represented by 4 features per channel. The vector size depends directly on the channel count:

NinaPro

8 channels $\times$ 4 features

= 32 dims / window

Custom Data

2 channels $\times$ 4 features

= 8 dims / window

Column names: RMS_ch1, MAV_ch1, WL_ch1, SSC_ch1, ... + Label. Feature computations are defined in a single shared module (emg_common.py).

MODELS

The Six Compared Models

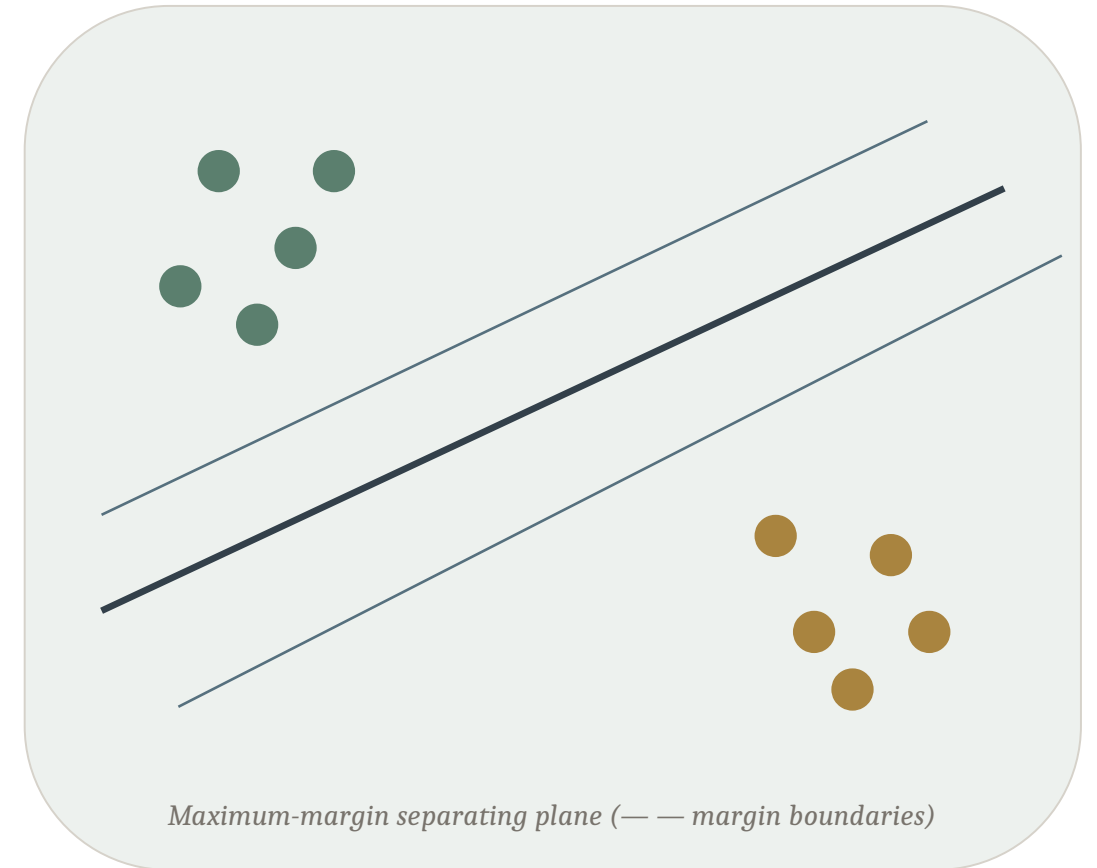


Two classical (SVM, RF), one ensemble-expert (MEET), one deep (CNN) and two baseline models (KNN, MLP).

How Does SVM Work?

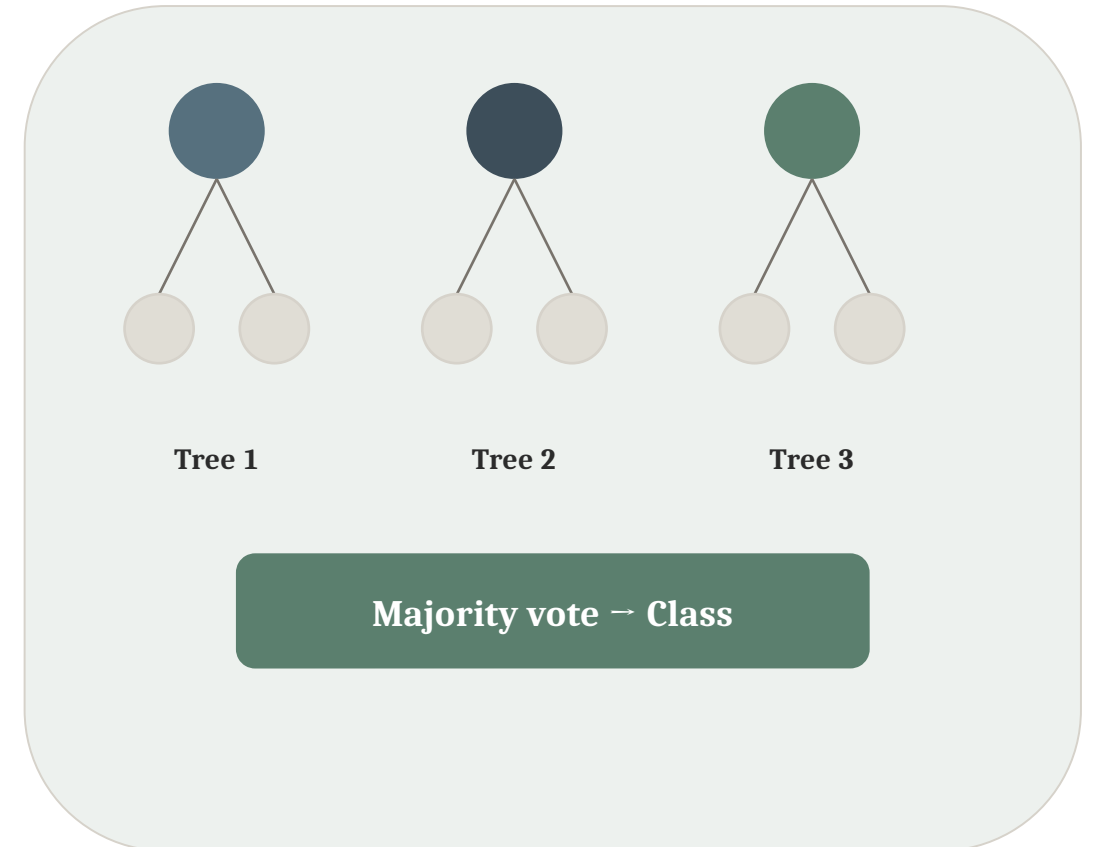
SVM looks for the decision boundary that separates two classes with the widest gap (margin).

- Maps non-linearly-separable data to a higher dimension via the RBF kernel
- There it finds the maximum-margin plane separating the two classes
- The samples nearest the boundary are the 'support vectors'; they define the boundary
- A new sample is assigned to whichever side of the plane it falls on



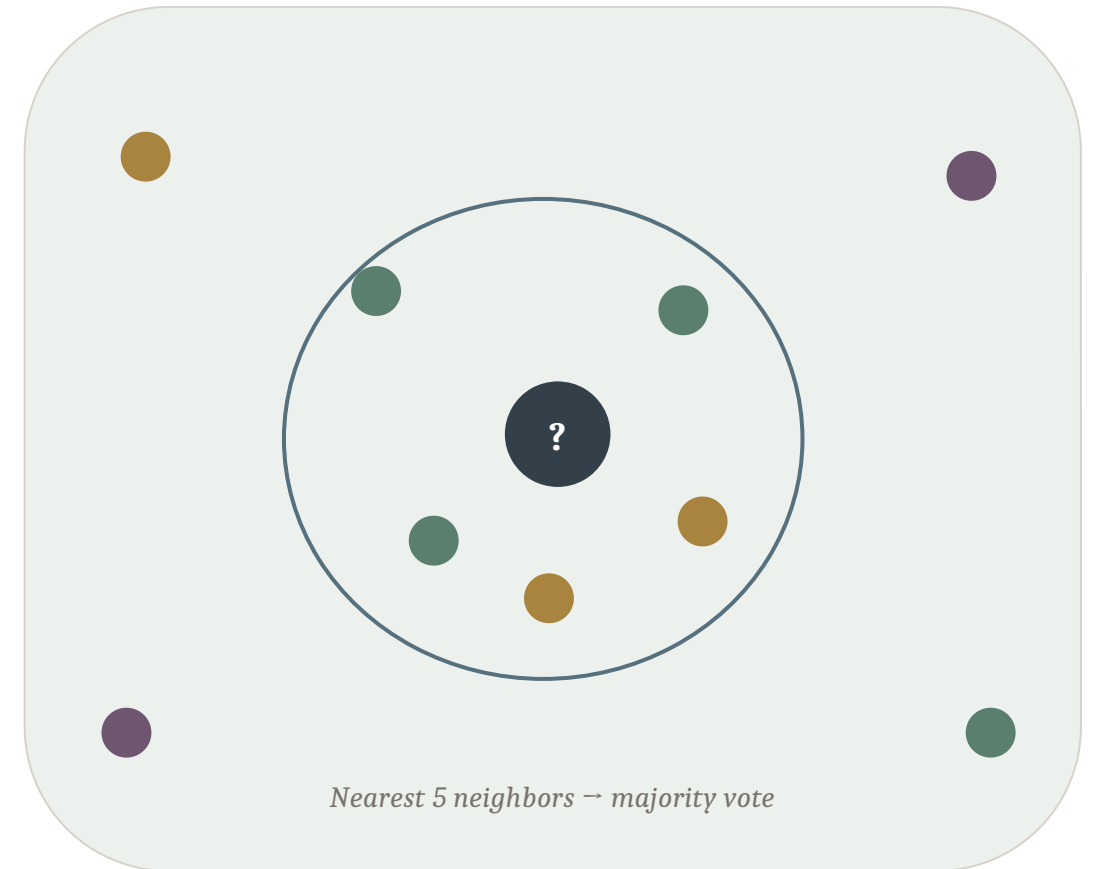
How Do Random Forest and MEET Work?

- Decision tree: splits the data with successive yes/no questions on features
- Random Forest: trains 100 different trees (random data + features)
- Each tree casts a vote → the majority decides the final class
- Extra Trees: also picks split thresholds randomly → lower variance
- MEET: a separate expert per class pair (One-vs-One), then votes are combined



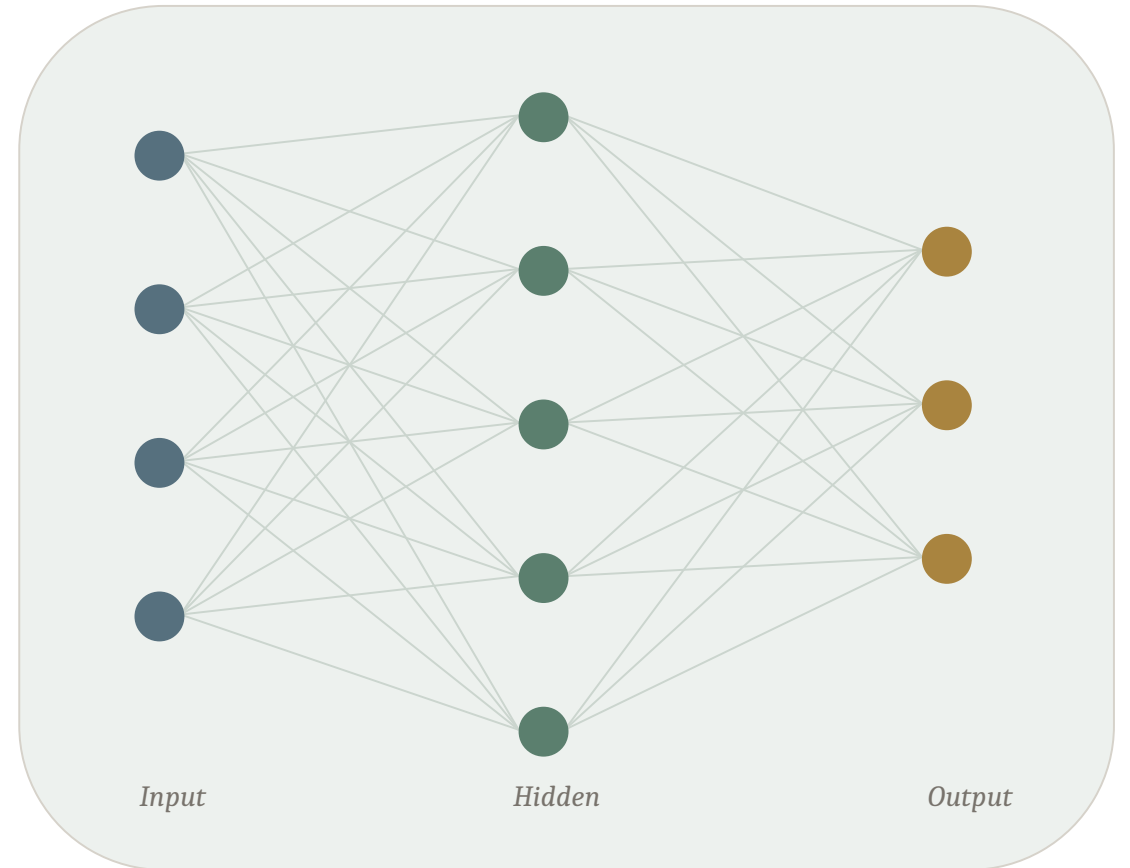
How Does KNN Work?

- There is no training; all samples are kept in memory
- When a new sample arrives, distances to all samples are computed
- The nearest $k = 5$ neighbors are selected
- The majority vote of these 5 neighbors gives the prediction



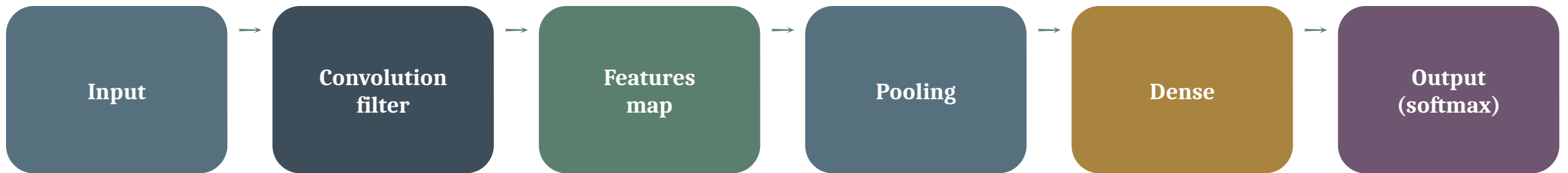
How Does the MLP (Neural Network) Work?

- Input: feature values (8 for the custom set)
- Hidden layer: each neuron is a weighted sum + ReLU activation
- Output layer: class probabilities via softmax
- Learning: weights are updated via backpropagation (backprop)



How Does the 1D CNN Work?

- Convolution (conv) filters slide over the input and capture local patterns
- ReLU + pooling: reduces size while keeping the important signal
- After flattening, a fully-connected (Dense) layer classifies; filters are learned during training



Raw/feature input → local pattern extraction → dimensionality reduction → classification

SVM — Support Vector Machine

SVM (RBF)

CHOSEN PARAMETERS

kernel	RBF (radial)
C	10
gamma	'scale'
random_state	22

WHY THIS CHOICE?

- RBF kernel: the EMG feature space is not linearly separable → a non-linear boundary is needed
- C=10: balances margin width vs. error penalty; a mid-high value that works well for sEMG
- gamma='scale': auto-scales the kernel width by feature variance
- Scaling is essential: SVM is distance-based and sensitive to standardization

Random Forest

Random Forest

CHOSEN PARAMETERS

<code>n_estimators</code>	100
<code>max_depth</code>	None
<code>n_jobs</code>	-1
<code>random_state</code>	22

WHY THIS CHOICE?

- 100 trees: enough for a stable ensemble; more adds marginal gain at higher cost
- `max_depth=None`: trees grow fully; the ensemble average offsets overfitting
- Bagging + random features → low variance, robust to noise
- Also provides feature importance; an interpretability advantage

MEET — Mixture of Experts Extra Trees

MEET (OvO + Extra Trees)

CHOSEN PARAMETERS

strategy	OneVsOne
base learner	ExtraTrees
n_estimators	100
random_state	22

WHY THIS CHOICE?

- OvO: a separate 'expert' per gesture pair $\rightarrow n(n-1)/2$ binary classifiers
- Separates hard pairs (B1 $\leftrightarrow$ B7, C5 $\leftrightarrow$ C14) more sharply than a single model
- Extra Trees: extremely randomized trees $\rightarrow$ lower variance, fast
- Adapted from Xiong et al. (2024)

KNN — k-Nearest Neighbors

KNN (k=5)

CHOSEN PARAMETERS

n_neighbors (k)	5
metric	Euclidean
training	none (lazy)
scaling	required

WHY THIS CHOICE?

- k=5: odd (no ties), small enough to capture local structure
- Euclidean distance: the vote of the 5 nearest neighbors on standardized features
- No training — all computation at prediction time; simple and interpretable
- Downside: sensitive to class overlap and noise; high memory/prediction cost

MLP — Multi-Layer Perceptron

MLP (1×128)

CHOSEN PARAMETERS

hidden layer	(128,) ReLU
output	softmax
solver	Adam, lr=0.001
max_iter	300 · seed 22

WHY THIS CHOICE?

- Single hidden layer, 128 neurons: enough capacity for a small feature space
- ReLU activation + softmax output → multi-class probability
- Adam optimizer: adaptive learning rate, fast convergence
- Learns non-linear decision boundaries; standardized input is essential

1D CNN — Two Architectures

NinaPro CNN (raw 150×8)

- 2× Conv1D (64/128) + BatchNorm + MaxPool
- Large kernels (10, 5) → temporal patterns
- Dense(256/128)+Dropout, output softmax(7)

Custom CNN (8 features)

- Conv1D(32,3) + MaxPool
- Dense(64)+Dropout, output softmax(6)
- No time axis → a tiny network suffices

Shared training: Adam (lr=0.001), sparse categorical crossentropy, early stopping (best-weight restore), ReduceLROnPlateau, balanced class weights, seed 22.

Evaluation Methodology

1

80% / 20% split

Stratified split preserving class balance

2

Scaling on training only

StandardScaler is 'fit' on training only → no data leakage

3

Within- & cross-subject

Same subjects / entirely unseen subjects (S1, S10)

4

Fixed seed (22)

Reproducibility across all models

5

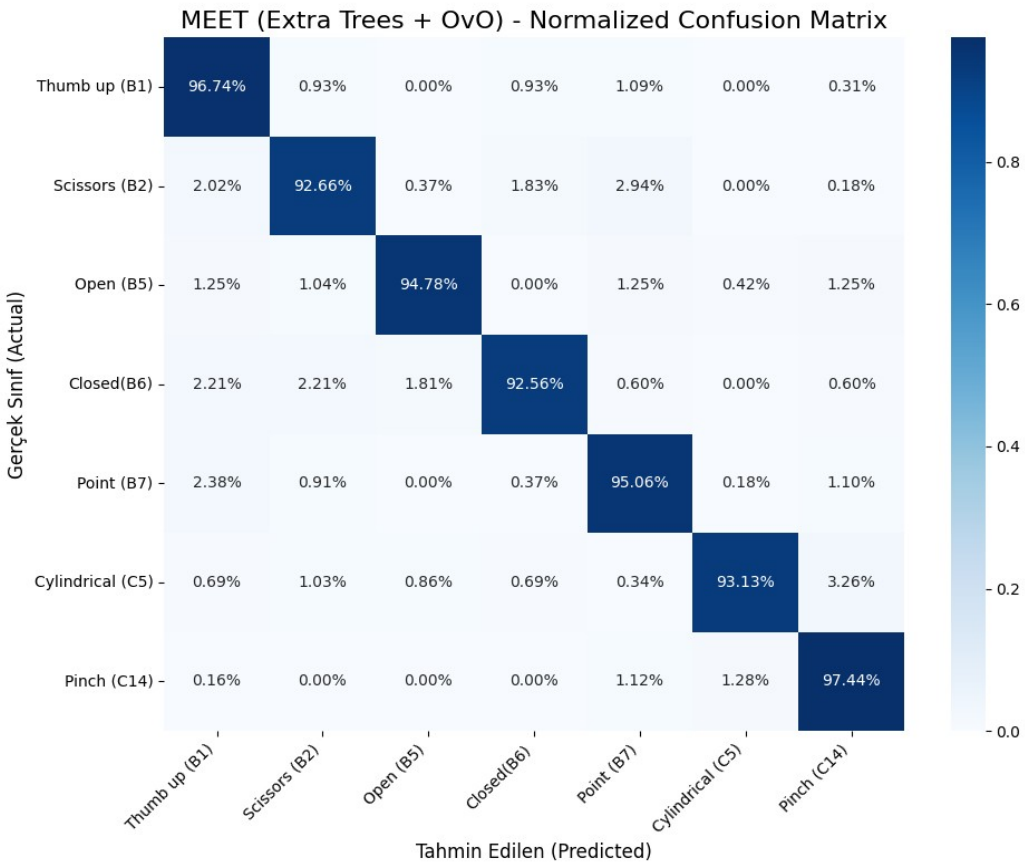
Metrics

Accuracy, precision/recall/F1, normalized confusion matrix

RESULTS

Results — NinaPro (6 Models)

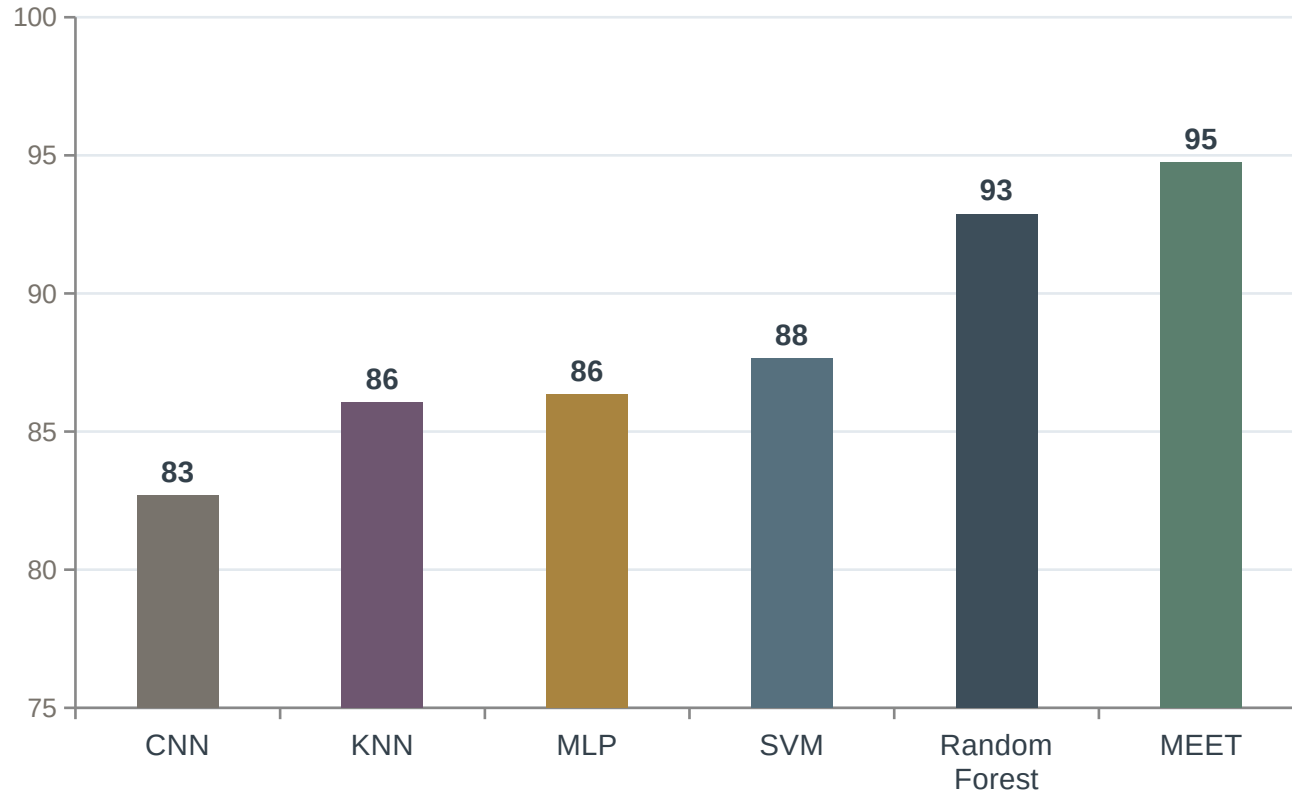
MEET	94.74%
Random Forest	92.88%
SVM (RBF)	87.65%
MLP (1×128)	86.35%
KNN (k=5)	86.06%
1D CNN (raw)	~ 82.7%



MEET — above 92% on almost every class

COMPARISON

NinaPro — Detailed Comparison



INTERPRETATION

- RF and MEET (ensemble/expert) are best by far
- SVM $\approx$ MLP $\approx$ KNN mid-group (~ 86 - 88%)
- CNN is last ($\sim 83\%$)
- Good features make the classical models strong

COMPARISON

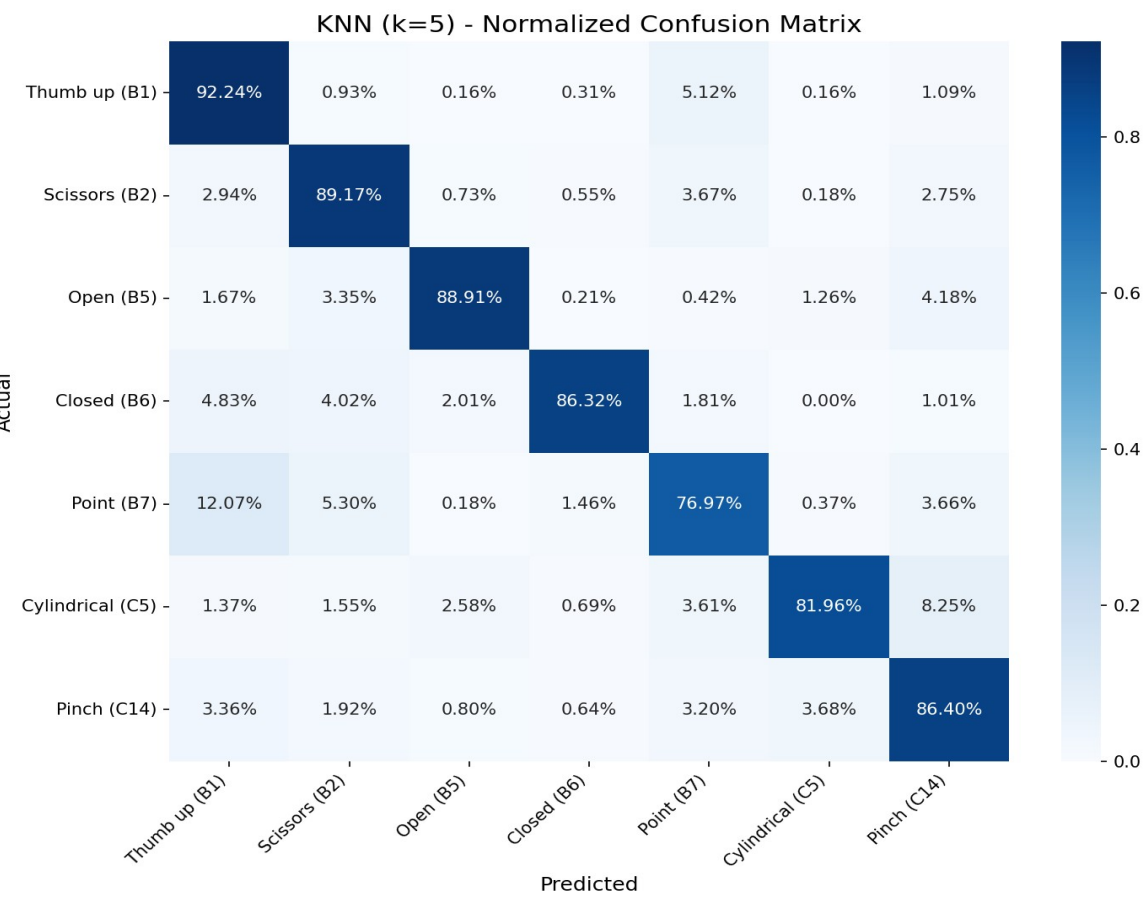
NinaPro — Per-Class Accuracy

Gesture	SVM	RF	MEET	KNN	MLP
Thumb up (B1)	87.4	93.9	96.7	92.2	91.9
Scissors (B2)	86.2	91.9	92.7	89.2	85.9
Open (B5)	90.2	94.6	94.8	88.9	86.2
Closed (B6)	88.3	89.9	92.6	86.3	83.3
Point (B7)	87.0	92.0	95.1	77.0	80.8
Cylindrical (C5)	84.4	90.4	93.1	82.0	85.4
Pinch (C14)	90.2	96.8	97.4	86.4	89.3

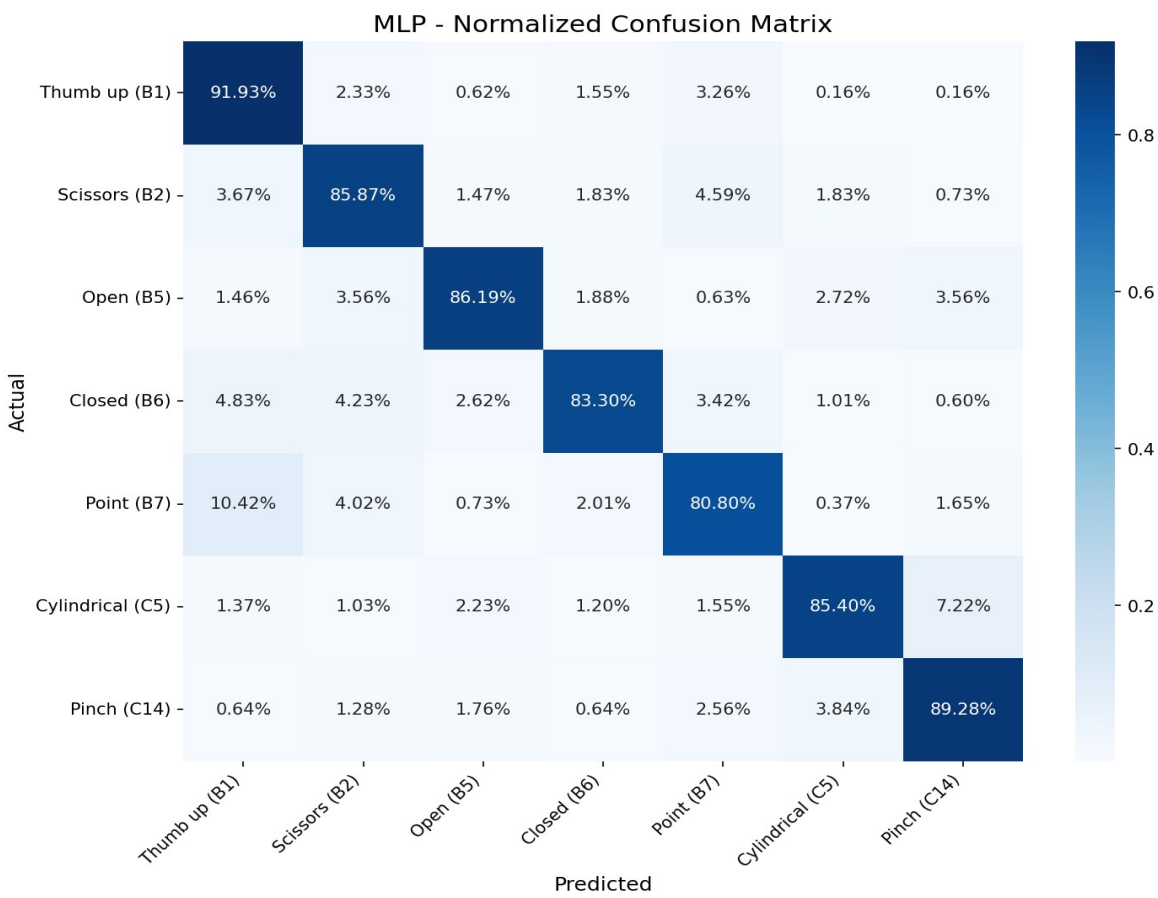
Values are % accuracy (recall). MEET leads on every class; KNN's weakest is Point (77%).

COMPARISON

NinaPro — KNN and MLP Matrices



KNN — Point/B7 is weakest (77%)



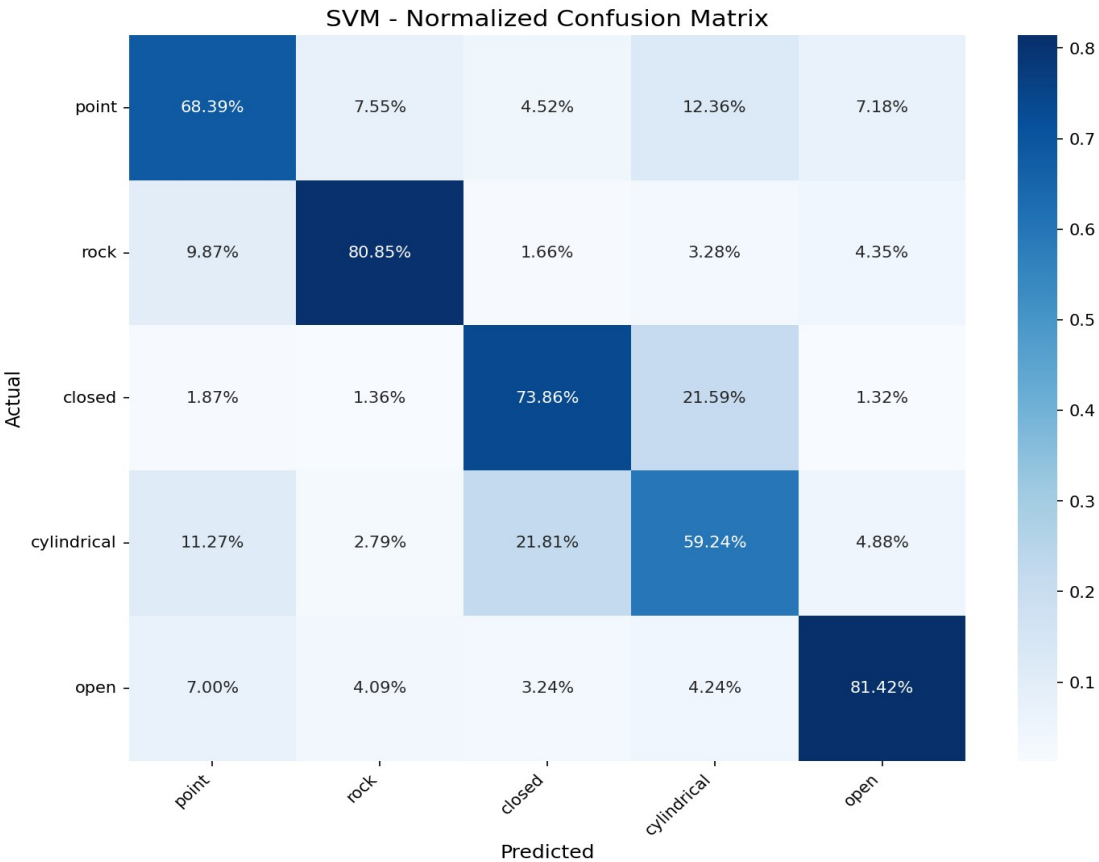
MLP — more balanced profile

RESULTS

Results — Custom Data (5 classes · no tip)

All models are in the ~70–73% band; KNN is weakest (70%).
Common bottleneck **cylindrical** ↔ **closed**.

Random Forest	72.98%
MLP (1×128)	72.92%
SVM (RBF)	72.75%
1B CNN	72.62%
MEET	72.46%
KNN (k=5)	69.68%



COMPARISON

Effect of Removing Cylindrical

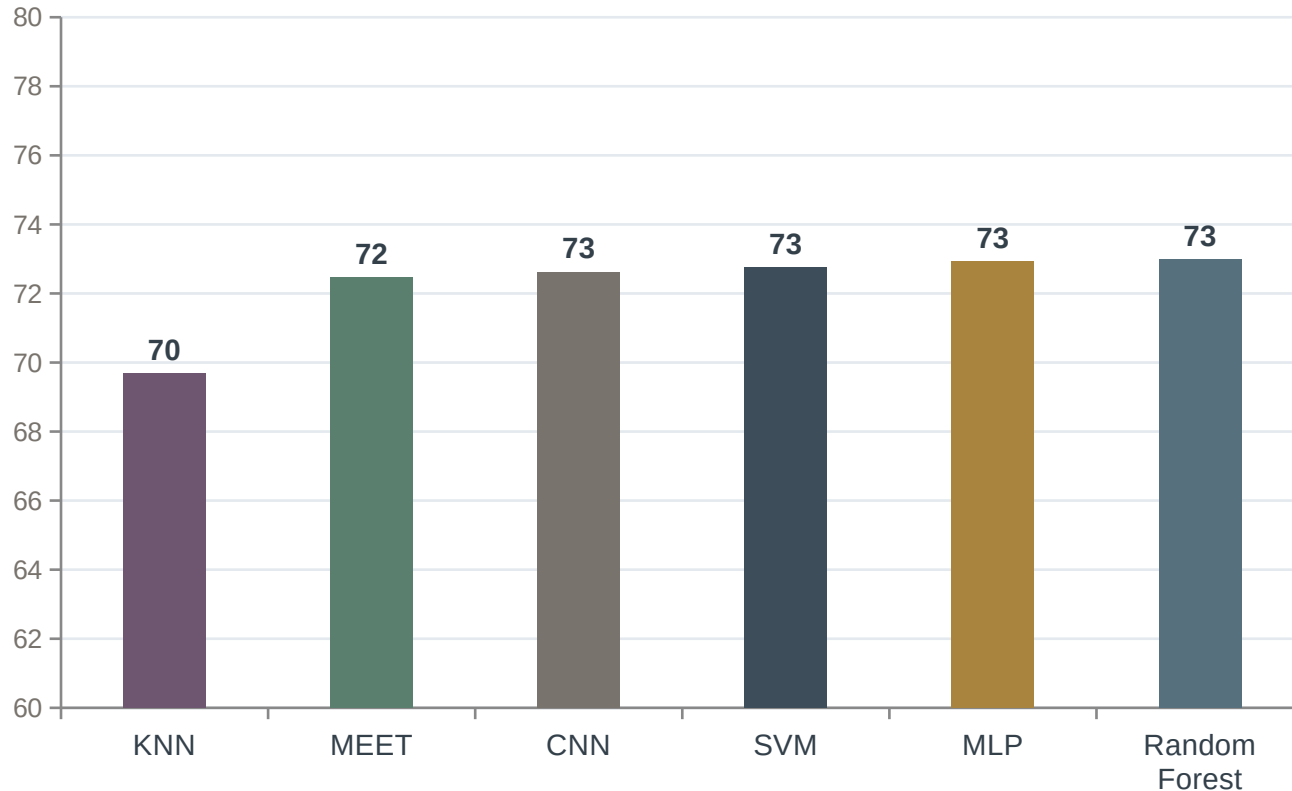
When cylindrical (the one inseparable class) is removed (4 classes) all models improve markedly. The biggest gain is in the 'closed' class:

Model	5 classes (with cyl)	4 classes (without cyl)	Change
KNN (k=5)	69.68%	81.50%	+11.8
MLP (1×128)	72.92%	83.21%	+10.3
1B CNN	72.62%	82.96%	+10.3
SVM / RF / MEET	~ 73%	~ 83%	~+10

'closed' class: KNN 72.5% → 89.4% · MLP 74.4% → 91.4% · CNN 69.3% → 91.7% (closed is ~92% for SVM/RF/MEET too). The cause is not the models but the inability to separate cylindrical-closed with 2 channels.

COMPARISON

Custom Data — 6-Model Comparison (with cyl)



INTERPRETATION

- All six models cluster around ~70–73%
- CNN (72.6%) is on par with the classical models
- KNN is lowest (70%) — fragile to overlap
- Common bottleneck: cylindrical ↔ closed

COMPARISON

Custom Data — Per-Class Accuracy (with cyl)

Gestures	SVM	RF	MEET	KNN	MLP	CNN
point	68.4	68.5	69.1	68.6	67.4	71.5
rock	80.9	82.2	81.3	78.5	82.5	80.6
closed	73.9	74.3	75.6	72.5	74.4	69.3
cylindrical	59.2	57.4	55.2	50.9	58.2	60.7
open	81.4	82.5	81.1	77.9	82.1	81.1

The cylindrical row is lowest across all six models (~51–61%) — the real bottleneck.

COMPARISON

Custom Data — With vs Without cyl

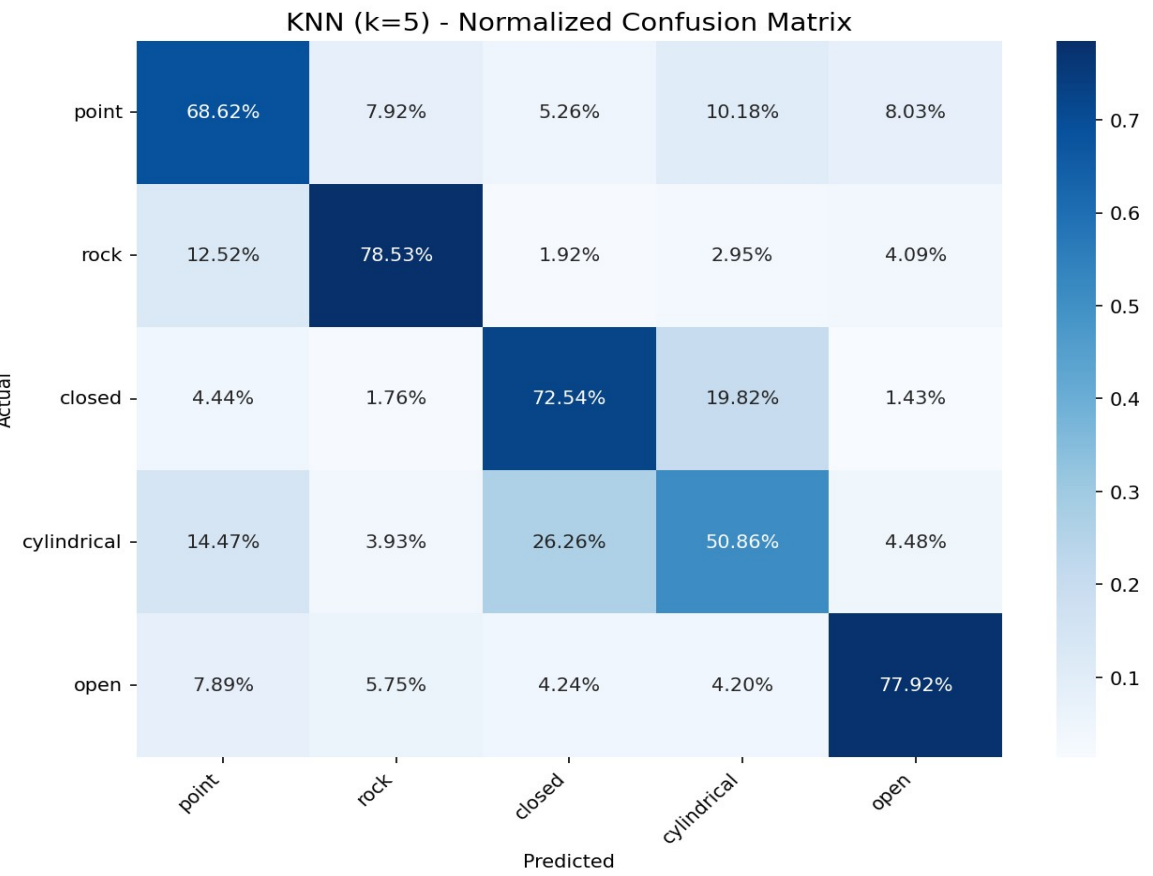


WHEN CYLINDRICAL IS REMOVED

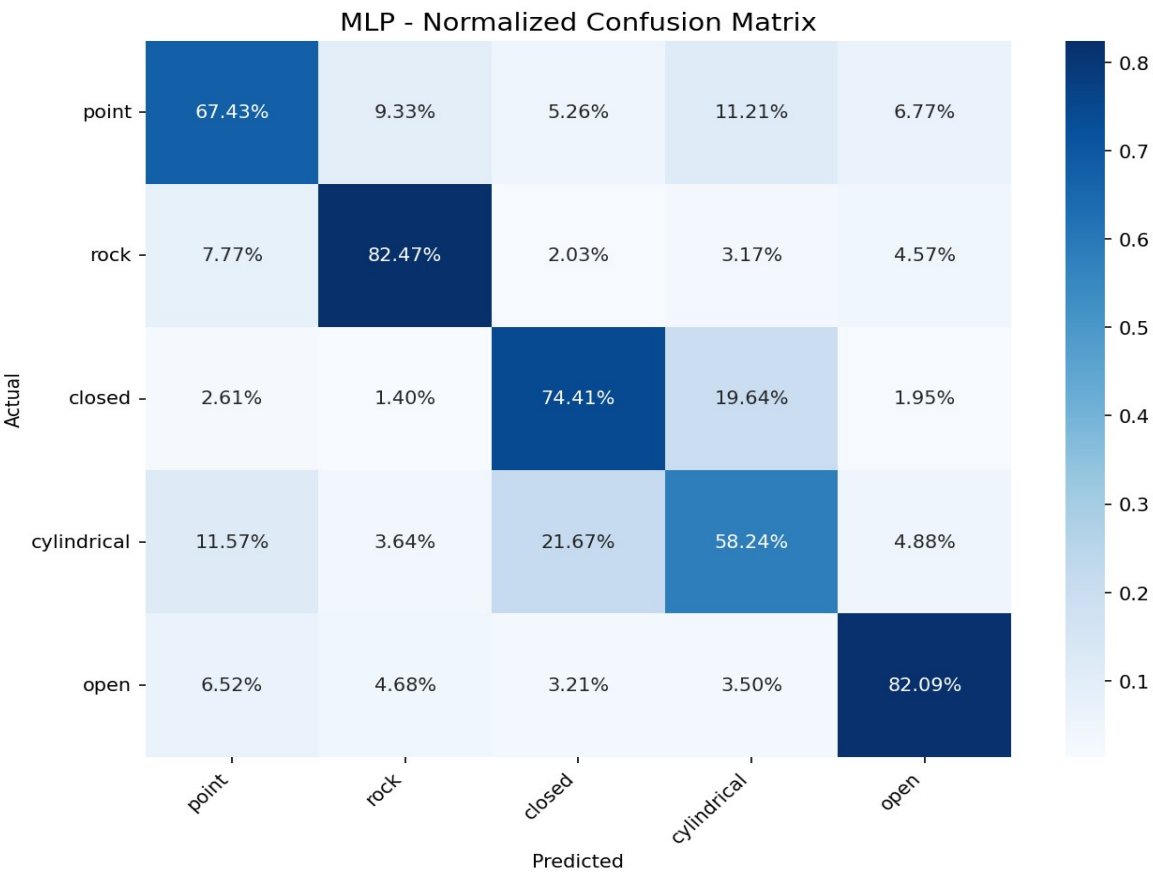
- KNN: 69.7% → 81.5% (+11.8)
- MLP: 72.9% → 83.2% (+10.3)
- CNN: 72.6% → 83.0% (+10.3)
- 'closed': KNN 73% → 89%, MLP/CNN 74% → 92%
- Classical (SVM/RF/MEET) also ~73% → ~83%

COMPARISON

Custom Data — KNN and MLP Matrices (with cyl)



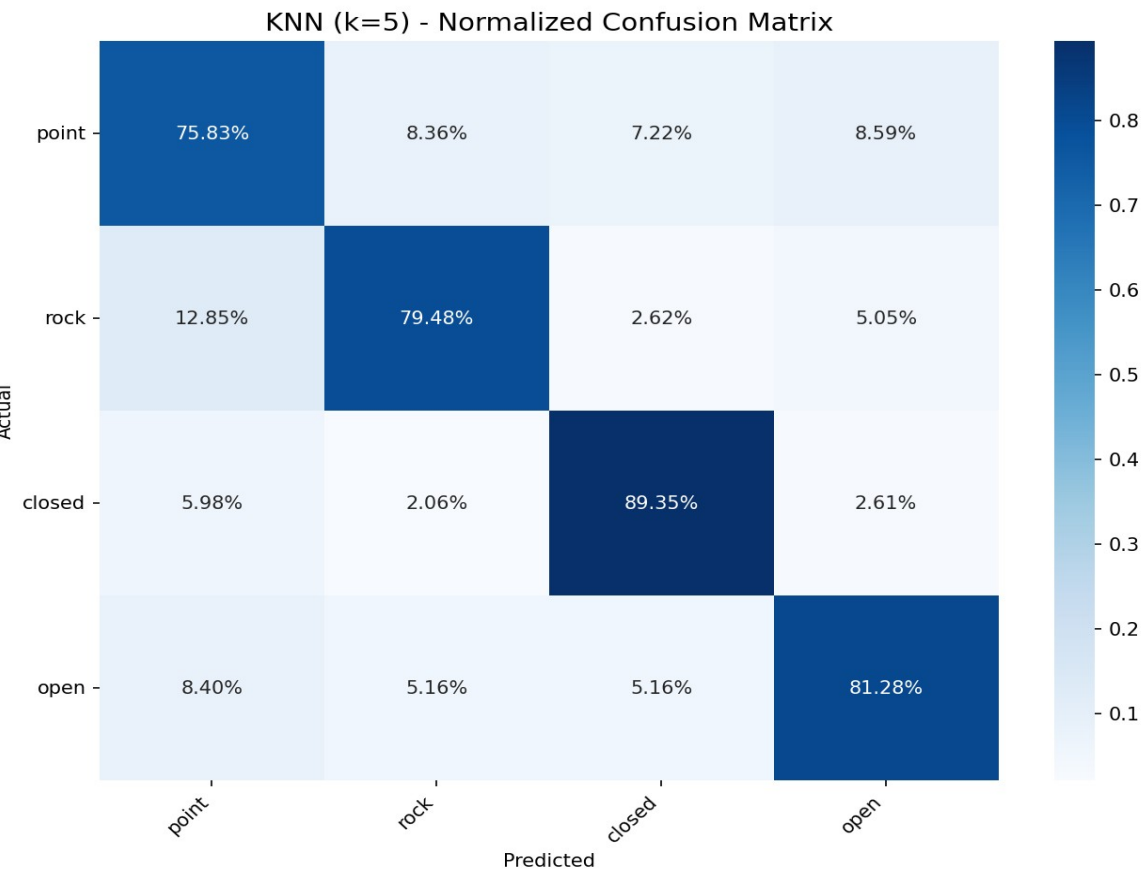
KNN — cylindrical 50.9%



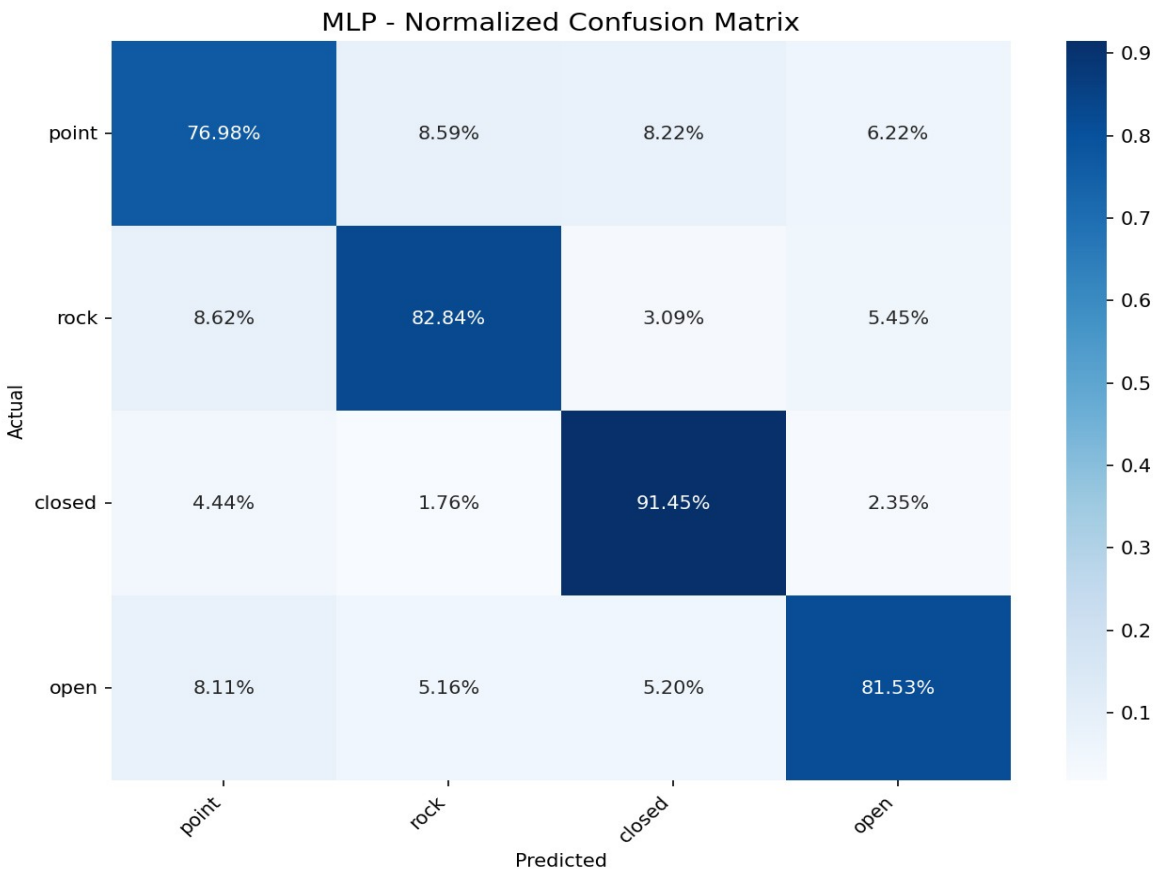
MLP — cylindrical 58.2%

COMPARISON

Custom Data — KNN and MLP Matrices (without cyl)



KNN — 'closed' rises to 89.4%



MLP — 'closed' rises to 91.4%

Comparative Analysis

Why NinaPro $\gg$ Custom data?

8 channels provide the spatial resolution needed to separate similar grips; 2 channels cannot.

Why are MEET / RF best?

Ensemble and One-vs-One expert structures separate hard pairs sharply and reduce variance.

MLP vs KNN?

The MLP competes with the classical models via its flexible boundary; KNN, being distance-based, is most fragile to overlap.

Why is the CNN behind?

At this data scale hand-crafted features are already strong; the CNN's raw-signal advantage is limited.

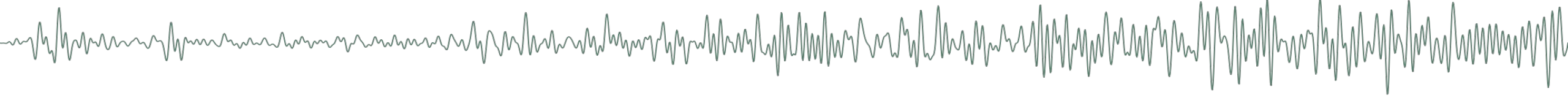
Challenges and Improvement Suggestions

Challenges Faced

- Cylindrical $\leftrightarrow$ closed confusion (main bottleneck)
- Spatial-information limit of 2 channels
- Reproducibility: seed inconsistency (fixed)
- Code duplication $\rightarrow$ consolidated into a shared module

Recommendations

- Increase channel count (highest return)
- More subjects $\rightarrow$ generalization
- Add frequency-domain features
- k-fold cross-validation
- MLP architecture / KNN distance-weight search



CONCLUSION

**Among the six models, MEET and Random Forest,
the best on 8-channel sEMG (> 92%)**

Well-designed time-domain features make the classical models strong; MLP is competitive, KNN the most basic. The ceiling on the custom data is not the model but the physical limit of 2 channels — the highest-return step is to increase the channel count.



Thank you • Questions?